

# Applied Data Science

T-786-APDS

## A Data Science Approach to Predicting Property Prices in Madrid

Davíð Rúnar Matthíasson

Iván Luque García

Julian Lorenz



May 26, 2026

# Contents

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                      | <b>1</b>  |
| 1.1      | Domain and problem setting . . . . .                                     | 1         |
| 1.2      | Research questions . . . . .                                             | 2         |
| 1.3      | Background and literature review . . . . .                               | 3         |
| <b>2</b> | <b>Description of the data</b>                                           | <b>3</b>  |
| 2.1      | Where does the data come from? How was it collected? . . . . .           | 3         |
| 2.2      | Exploratory data analysis . . . . .                                      | 4         |
| <b>3</b> | <b>Data Analysis</b>                                                     | <b>7</b>  |
| 3.1      | Data Preprocessing . . . . .                                             | 7         |
| 3.2      | Post-Split Processing . . . . .                                          | 7         |
| 3.2.1    | Orientation Flag Imputation ( <code>fill_orientation</code> ) . . . . .  | 7         |
| 3.2.2    | Amenity Imputation by Cohort ( <code>fill_amenities</code> ) . . . . .   | 8         |
| 3.2.3    | Feature Name Cleaning ( <code>clean_feature_names</code> ) . . . . .     | 8         |
| 3.2.4    | Column Curation ( <code>curate_df</code> ) . . . . .                     | 8         |
| 3.2.5    | Data Preparation Pipeline ( <code>prepare</code> function): . . . . .    | 8         |
| 3.3      | Train/test split . . . . .                                               | 8         |
| 3.4      | Anomaly Detection on Train set . . . . .                                 | 9         |
| 3.5      | Log-Transformation of Skewed Target Distribution . . . . .               | 9         |
| 3.6      | ML algorithms . . . . .                                                  | 10        |
| <b>4</b> | <b>Evaluation Methodology</b>                                            | <b>11</b> |
| 4.1      | Choice and Justification of Performance Metrics . . . . .                | 11        |
| 4.1.1    | Coefficient of Determination ( $R^2$ ) . . . . .                         | 12        |
| 4.1.2    | Mean Absolute Error (MAE) . . . . .                                      | 12        |
| 4.1.3    | Root Mean Squared Error (RMSE) . . . . .                                 | 12        |
| 4.1.4    | Median Absolute Error (MedAE) . . . . .                                  | 13        |
| 4.1.5    | Mean Absolute Percentage Error (MAPE) . . . . .                          | 13        |
| 4.1.6    | Metrics Summary . . . . .                                                | 13        |
| <b>5</b> | <b>Performance and Results</b>                                           | <b>13</b> |
| 5.1      | Success Criteria . . . . .                                               | 13        |
| 5.2      | SHAP-based feature importance . . . . .                                  | 14        |
| 5.3      | Performance and Model interpretation . . . . .                           | 16        |
| 5.4      | Performance Border and Domain Constraints . . . . .                      | 17        |
| 5.5      | Impact of Log-Transformation on Model Performance . . . . .              | 18        |
| 5.6      | Training Stability and Convergence Analysis . . . . .                    | 19        |
| 5.7      | Data Size Effects on Model Architecture (Boosting and Bagging) . . . . . | 20        |
| 5.8      | Role of district in price predictions . . . . .                          | 20        |
| 5.9      | Statistical testing via Paired t-tests . . . . .                         | 20        |
| 5.10     | Answering the research questions . . . . .                               | 22        |
| <b>6</b> | <b>Ethical Considerations</b>                                            | <b>22</b> |
| 6.1      | Data Bias . . . . .                                                      | 22        |
| 6.2      | The Five C's . . . . .                                                   | 23        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>7</b> | <b>Conclusion</b>                                             | <b>24</b> |
| <b>A</b> | <b>Appendix: Web simulation of the asking-price estimator</b> | <b>27</b> |
| A.1      | Purpose and scope . . . . .                                   | 27        |
| A.2      | Architecture . . . . .                                        | 27        |
| A.3      | Feature reconstruction at inference time . . . . .            | 28        |
| A.4      | User interaction, uncertainty, and limitations . . . . .      | 28        |

## List of Figures

|    |                                                                                                                                                                                                                                                                                                                                               |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Histogram of numerical features . . . . .                                                                                                                                                                                                                                                                                                     | 5  |
| 2  | buy_price - Raw and Log-transformed . . . . .                                                                                                                                                                                                                                                                                                 | 6  |
| 3  | Madrid real estate exploratory data analysis: (top-left) relationship between built surface and price, (top-right) the ten features most correlated with buy_price, (bottom-left) median price for the ten districts with the highest median asking price, and (bottom-right) the fifteen features with the highest missingness rate. . . . . | 6  |
| 4  | Price distribution by district (listings under € 5M shown). Within-district spread increases sharply with median price: the cheapest districts have both lower prices and tighter distributions, while the most expensive districts contain a wide mix of prices ranging from under € 500,000 to well over € 3M. . . . .                      | 7  |
| 5  | Shap Global Importance . . . . .                                                                                                                                                                                                                                                                                                              | 14 |
| 6  | Shap Beeswarm . . . . .                                                                                                                                                                                                                                                                                                                       | 15 |
| 7  | Training convergence comparison: raw (left) vs log-transformed (right) target scales. Blue and orange lines represent independent validation folds. Log-scale training exhibits tighter fold agreement and earlier convergence plateau, indicating more stable regularisation and reduced variance across CV splits. . . . .                  | 19 |
| 8  | The importance of districts as an error correction feature . . . . .                                                                                                                                                                                                                                                                          | 20 |
| 9  | Paired t-test on regression models . . . . .                                                                                                                                                                                                                                                                                                  | 21 |
| 10 | Paired t-test on regression models with tuned hyperparameters . . . . .                                                                                                                                                                                                                                                                       | 22 |
| 11 | Web simulation of the asking-price estimator, running locally in the browser . . . . .                                                                                                                                                                                                                                                        | 27 |

# 1 Introduction

Predicting residential property prices is a common applied problem in data science because housing prices depend on many interacting factors, including property size, location, amenities, building characteristics, and market conditions. Although the task can be framed simply as estimating a single monetary value from a set of attributes, the underlying price-formation process is considerably more complex. Two properties with similar physical characteristics may have very different prices depending on their district, neighbourhood, condition, or perceived desirability.

This project studies the use of supervised machine learning for predicting residential property asking prices in Madrid. The objective is not to replace professional appraisal, but to evaluate how well a data-driven model can approximate listed prices using the structured information available in online property advertisements. Since the available target variable corresponds to asking prices rather than final transaction prices, the model should be interpreted as a predictor of advertised prices, not as a definitive estimate of true market value.

The project follows an end-to-end applied data science workflow. It includes data exploration, data cleaning, leakage prevention, preprocessing, model comparison, hyperparameter tuning, final evaluation, and model interpretation. In addition to predictive performance, the report also considers whether the final model behaves in a way that is consistent with real estate intuition. A useful model should not only produce accurate estimates, but should also rely on meaningful features such as built surface, location, number of bathrooms, property type, and amenities.

The remainder of this chapter introduces the domain and problem setting, defines the research questions, and reviews the methodological background relevant to the project. As an addition to this core modeling work, Appendix A documents a local web simulation built around the saved model, illustrating what a deployment of the estimator might look like in practice.

## 1.1 Domain and problem setting

Residential property valuation is a supervised regression problem in which the target variable is a continuous price and the predictors describe the observable characteristics of each property. In this project, each observation corresponds to one residential property listing in Madrid, and the objective is to estimate its listed asking price in euros from the information available in the advertisement.

The dataset used in this project is the *Madrid Real Estate Market* dataset, published on Kaggle (17). It was collected by crawling popular real estate portals in Madrid over an 18-day period in 2020, from 10 to 28 of March. The scraping process resulted in 21,742 property listings across the Madrid metropolitan area, each described by 58 attributes covering location, structural characteristics, amenities, and listing prices. The dataset is therefore a cross-sectional snapshot of the Madrid residential market at a specific point in time, capturing properties that were actively advertised for sale during that collection window.

The dataset contains a mixture of numerical, categorical, and binary variables. Numerical variables include property size, number of rooms, number of bathrooms, construction year, and listed price. Categorical variables include property type and location-related information, while binary variables describe amenities such as lift availability, parking, air conditioning, furnished status, and orientation flags. This mixed structure makes the dataset representative of a realistic tabular machine learning problem, where different preprocessing strategies are required for different feature types.

A central limitation of this setting is that the recorded prices are asking prices set by sellers, not final transaction prices from a public registry. This introduces a degree of selection bias toward properties that were actively on the market during the collection period. It also means that the target variable should be interpreted as the price at which a property was offered, not necessarily the price at which it would eventually be sold. For this reason, the report refers to the prediction target as an asking price, listed price, or advertised price, rather than as a definitive market value.

The price variation observed in the dataset is substantial. Asking prices range from roughly €36,000 for the cheapest listing to approximately €8.8 million for the most expensive, with a median around €375,000. This variation is not random. Built surface is expected to be one of the strongest predictors of price, but properties with similar surface area can still be advertised at very different prices depending on their district or neighbourhood. Central and high-demand areas command a substantial premium over peripheral districts, which makes location one of the key challenges in the modelling task.

The raw dataset also presents several common challenges in applied data science. Some variables have high missingness, some columns are redundant or near-duplicate measures of the same concept, and some price-derived variables would introduce target leakage if used during modelling. For example, variables derived from price per square metre cannot be used as predictors when the goal is to estimate the price itself. These issues justify the construction of a complete machine learning pipeline that includes data cleaning, leakage prevention, preprocessing, train-test splitting, model selection, final evaluation, and model interpretation.

Reliable price estimates matter to several groups of stakeholders. Buyers and sellers can benefit from an evidence-based reference point when negotiating, rather than relying only on individual judgement. Real estate agencies and investors can use automated estimates to screen listings and identify potentially over-priced or under-priced properties. However, because the model is trained on advertised prices rather than completed sales, its output should be treated as a decision-support estimate rather than as a replacement for professional appraisal. In-person appraisal can incorporate qualitative information that is not present in the dataset, such as interior condition, street-level desirability, views, noise, renovation quality, or seller urgency. At the same time, professional appraisal can be slow, costly, and subject to variation between appraisers, which motivates a complementary machine-learning approach.

## 1.2 Research questions

This project is guided by the following research questions:

1. Which machine learning model achieves the best predictive performance for estimating Madrid residential asking prices from structured listing data?
2. How much does the best-performing model improve over simpler baseline models, and is the improvement large enough to justify the additional model complexity?

These questions are designed to evaluate both predictive performance and practical reliability. The goal is not only to minimise error, but also to understand why the model performs as it does and where its limitations come from. These questions will be answered in Section 5.10.

### 1.3 Background and literature review

Residential property valuation has been studied across urban economics, real estate appraisal, and applied statistics, with each tradition emphasising different criteria. Economic theory seeks coefficients that quantify how attributes contribute to price, professional appraisal seeks defensible per-listing estimates, and data-driven valuation seeks accurate predictions on unseen properties at scale. The three traditions overlap in subject matter but not in evaluation criteria, which is one reason a project of this kind draws on all of them.

The economic foundation for predicting prices from observable attributes is the hedonic framework of Rosen (16), which decomposes the price of a heterogeneous good into the contributions of its characteristics. Under this view, a regression of listed price on structural, location, and amenity features is the empirical analogue of the underlying economic model. In their survey of operational valuation methods, Pagourtzi et al. (14) identify two issues that bear directly on the present work: real estate markets are heterogeneous in ways no structured dataset captures in full, and professional appraisal is accurate but slow and subject to inter-appraiser variance. The first issue sets a lower bound on the accuracy achievable by any algorithm trained on observable features alone, and the second motivates data-driven valuation as a complementary decision-support tool rather than as a replacement for human expertise.

A further distinction the literature draws is between the price at which a property is offered and the price at which it is eventually transacted. Levitt and Syverson (11) show that this gap is systematic rather than random and is driven by information asymmetries between sellers, buyers, and agents. The dataset used here records asking prices only, so the model is trained to predict what sellers offer rather than what buyers settle, and the report refers to the target accordingly throughout.

On the methodological side, the project follows standard practice for medium-sized, mixed-type tabular regression. Preprocessing is fitted on the training partition only to prevent leakage, model selection is conducted through nested cross-validation to obtain an unbiased estimate of generalisation error in the sense of Raschka (15), and the comparison of model families includes both regularised linear baselines and tree-based ensembles, with the latter expected to dominate when feature interactions matter, as confirmed in Section 5. Model interpretation is conducted post-hoc through feature attributions rather than through linear coefficients, in line with the gradient-boosted final model.

## 2 Description of the data

This chapter introduces the dataset used throughout the project. We carry out an exploratory data analysis of missingness, target distribution, and the geographic price structure across Madrid. The findings motivate the preprocessing decisions in the next chapter.

### 2.1 Where does the data come from? How was it collected?

Data was collected through crawling popular real estate portals in Madrid. Each row in the dataset corresponds to a single property listing in Madrid.

The prices recorded are the asking prices that are set by the sellers of the real-estate. It is not a real transaction price from a public registry. This introduces a degree of selection bias toward properties actively on the market and means the target variable should be interpreted as the

price at which a property was offered, not the price at which it was sold.

In its raw form the dataset contains 21,742 listings by 58 columns. Some of the most important variables, which we will look further into in Section 5.2, are described in Table 1.

| Variable                     | Description                                                                                                                                                                                                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>buy_price</code>       | Listed asking price in €. Target of the regression task.                                                                                                                                                                                                                                    |
| <code>sq_mt_built</code>     | Built surface area of the property in square metres. The single strongest predictor of price.                                                                                                                                                                                               |
| <code>n_rooms</code>         | Number of rooms in the property.                                                                                                                                                                                                                                                            |
| <code>built_year</code>      | Year the building was constructed.                                                                                                                                                                                                                                                          |
| <code>house_type_id</code>   | Categorical label for the type of dwelling (flat, penthouse, duplex, studio, and so on).                                                                                                                                                                                                    |
| <code>neighborhood_id</code> | Location string of the form “Neighborhood 133: Los Rosales (1827.79 €/m2) - District 21: Villaverde”, embedding both a district number and a finer neighborhood number. The district number is later extracted as <code>district_id</code> and used as the geographic feature in the model. |

Table 1: Main variables retained from the raw dataset.

A handful of columns (identifiers, raw addresses, and rent-derived prices) are administrative or would introduce target leakage, and are removed before modelling.

The target variable for the regression task carried out in the following sections is `buy_price`, expressed in euros. The dataset is released under a permissive license and contains no personally identifying information beyond street-level addresses.

## 2.2 Exploratory data analysis

A first inspection of the raw dataframe shows that data quality varies considerably across columns. Several variables exhibit missingness rates above 50%, some even 100%, which motivates dropping them rather than relying on aggressive imputation, since features that sparse carry little reliable signal. A representative example is `n_floors`, the number of floors of the listing: in the small fraction of listings where it is recorded, the value is almost always greater than one, so the column does carry some information in principle. However, because the variable is missing for the large majority of listings, any imputation strategy would essentially fabricate the value for most of the dataset, and the column is therefore dropped together with the other high-missingness variables (Figure 3, bottom-right). Among the retained columns, missingness is more moderate and can be addressed through targeted imputation strategies discussed in Section 3. A histogram was made of the numerical features to aid us in the process of deciding what to do with these columns. Histograms are shown in Figure 1.



Figure 1: Histogram of numerical features

The correlation analysis reveals pairs of near-duplicate numeric features (for example surface measured in slightly different ways), which are reduced to a single representative variable to avoid redundancy and collinearity.

The target variable `buy_price` is strongly right-skewed, with a long tail driven by a small number of luxury properties. A logarithmic transformation yields a distribution that is much closer to symmetric (Figure 2), which is preferable for most regression algorithms and is the form used during model training. Two further patterns visible in Figure 3 are worth noting. The scatter of price against built surface (top-left) confirms a strong, near-linear relationship between the two, with an  $R^2$  of 0.692 for a simple univariate fit — consistent with the correlation ranking that places `sq_mt_built` and `n_bathrooms` as the two features most associated with the target. The median price by district (bottom-left) shows substantial variation across Madrid: the median asking price in the most expensive of the top ten districts is more than three times that of the cheapest among the top 10. This justifies retaining `district_id` as a categorical feature.



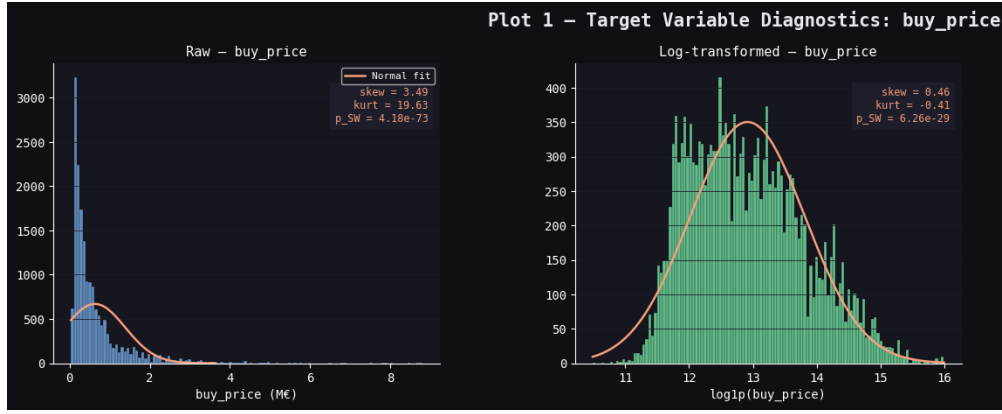


Figure 2: buy\_price - Raw and Log-transformed

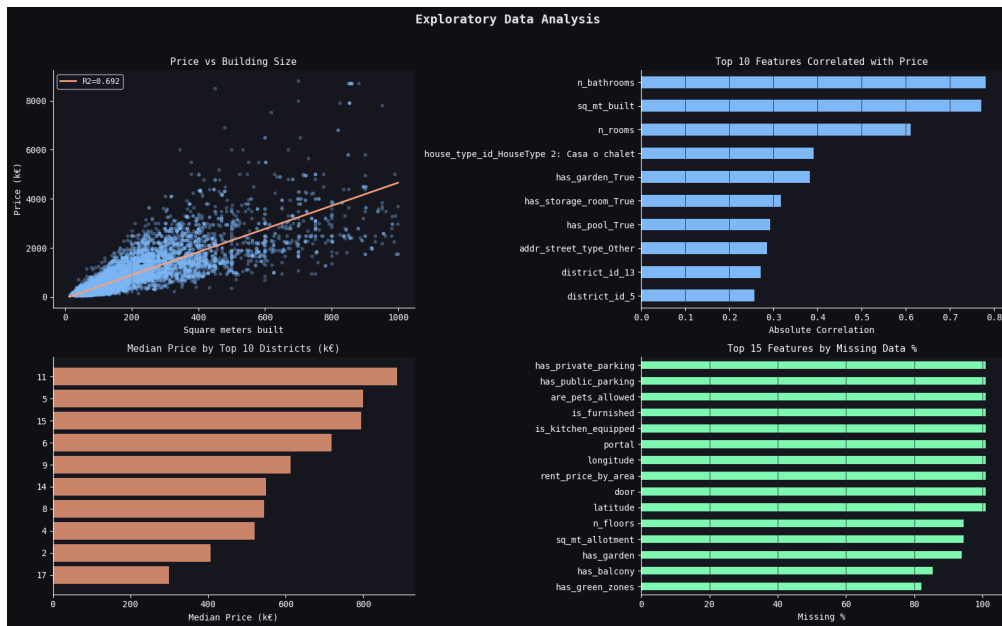


Figure 3: Madrid real estate exploratory data analysis: (top-left) relationship between built surface and price, (top-right) the ten features most correlated with `buy_price`, (bottom-left) median price for the ten districts with the highest median asking price, and (bottom-right) the fifteen features with the highest missingness rate.

A more detailed view of the same pattern is given in Figure 4, which shows the full price distribution within each district. Within-district price variability grows strongly with the district's median price: the cheapest districts have both lower prices and tighter distributions, while the most expensive districts span more than €1M between their first and third quartiles, with long upper tails reaching well above €3M. The high-median districts are therefore heterogeneous rather than uniformly expensive, which means a single `district_id` dummy will capture an average effect rather than a precise location premium.

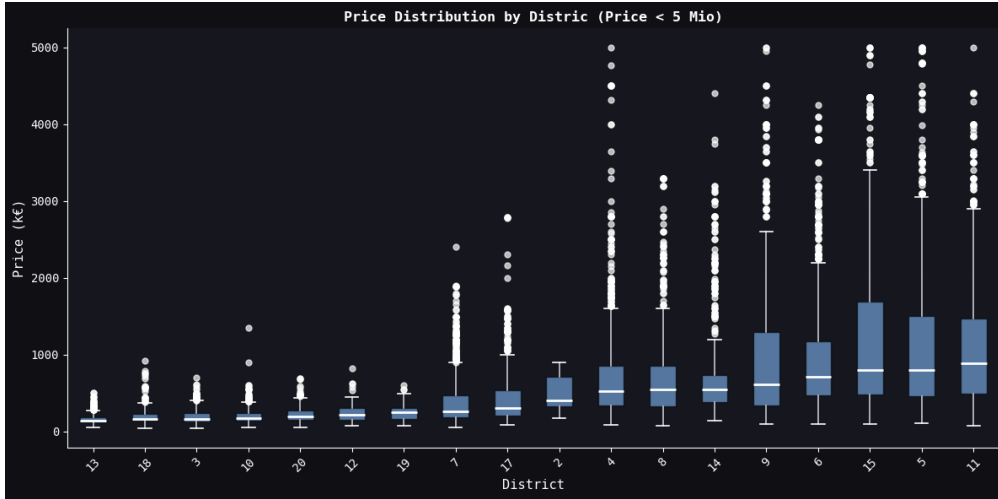


Figure 4: Price distribution by district (listings under €5M shown). Within-district spread increases sharply with median price: the cheapest districts have both lower prices and tighter distributions, while the most expensive districts contain a wide mix of prices ranging from under €500,000 to well over €3M.

### 3 Data Analysis

This section documents the data-analysis pipeline: the preprocessing steps applied to the raw dataset, the train/test partition, and the precautions taken against information leakage, anomaly detection, and the five machine learning models compared in the remainder of the report.

#### 3.1 Data Preprocessing

The raw Kaggle dataset contained data inconsistencies, missing values, feature redundancy and data leakage that required systematic remediation prior to model training. We adopt the principle that preprocessing quality directly determines model performance (7). Overall a one-step pre-split processing and a five-step post-split pipeline was applied, each addressing a specific category of data quality issue. For pre-split processing only, duplication dropping was used as it is important that no unintended data leakage would appear. The next section is going to explain the applied post-split processing.

#### 3.2 Post-Split Processing

Following the train-test partition at 80% / 20%, all statistical transformations (scaling, outlier clipping, variance thresholding) were fit exclusively on the training set to prevent information leakage. When applied to the test set, these fitted transformers used training-set statistics to ensure that test-set predictions reflect the model’s behaviour on truly unseen data. Feature selection via correlation thresholding and variance filtering occurred after the split, on the training set only, ensuring the test set remained completely isolated from architecture and hyperparameter decisions.

##### 3.2.1 Orientation Flag Imputation (fill\_orientation)

Missing values in binary orientation flags (“faces\_north”, “faces\_south”, etc.) were filled with the global mode (most frequent class) across the entire dataset, as construction year provides no reliable signal for predicting orientation. Unlike amenities, which correlate strongly with

era, building orientation is determined by plot geography and architectural design rather than temporal trends. The global mode approach is justified because orientation missingness is likely missing-at-random (reflecting incomplete listing data) rather than missing-by-design, and the mode respects the observed marginal distribution of orientation in the market.

### 3.2.2 Amenity Imputation by Cohort (`fill_amenities`)

Missing values in boolean amenity flags (e.g., `has_lift`, `has_ac`) were imputed with False rather than via statistical methods (e.g., mode, cohort median). This choice reflects the information structure of real estate listings: property attributes that exist are typically recorded by real estate agents to highlight market value, while absence of mention indicates absence of the amenity. Treating missing as False is thus a conservative assumption. Furthermore, no False values were observed in the raw data (only True or NaN), confirming that listings never explicitly record the absence of amenities; they either mention them (True) or do not (NaN = implicitly False).

### 3.2.3 Feature Name Cleaning (`clean_feature_names`)

This step replaces illegal characters with underscores and collapses consecutive underscores, rendering names like “house\_type\_id\_HouseType\_2\_Casa\_o\_chalet” from “house\_type\_id\_HouseType 2: Casa o chalet”. While XGBoost and GradientBoosting are more lenient, cleaning ensures compatibility with all gradient boosting frameworks without requiring framework-specific workarounds.

### 3.2.4 Column Curation (`curate_df`)

Columns with missing data exceeding a specified threshold (default: 60%) were removed entirely from the dataset. This threshold reflects the principle that features with majority-missing values carry minimal predictive signal and would require a lot of imputation that introduces systematic bias. For the Madrid dataset, 23 columns fell below the threshold and were dropped, including rarely-recorded features such as “n\_floors” (92.6% missing) and “latitude” (100% missing), which could not be reliably inferred from observed data.

### 3.2.5 Data Preparation Pipeline (`prepare function`):

The `prepare` function executes a sequence of 13 preprocessing steps. E.g. it (1) drops administrative columns (IDs, URLs, listing dates); (2) enforces non-null square footage and bathroom counts (core structural features); (3) fills boolean amenity flags with False (as a backup); (4) imputes construction year using district-level median with global median fallback; (5) extracts the `district_id` from the raw `neighborhood_id` string via regex and then drops `neighborhood_id`, while retaining `subtitle`, the barrio name string, as a separate high-cardinality categorical; (6) encodes the raw address into three features, namely a street-type categorical, a binary flag for whether a street number is present, and a bucketed numeric feature for the magnitude of the number; (7) applies one-hot encoding with `drop_first=True` to all surviving categorical columns, namely `subtitle`, `district_id`, `house_type_id`, `floor`, `energy_certificate`, and `addr_street_type`.

## 3.3 Train/test split

Before any modelling, the curated dataset is partitioned into a training set and a held-out test set using a standard random 80/20 split with a fixed random seed (`random_state=42`). The

fixed seed ensures the split is reproducible across runs and avoids the optimistic results that can arise when an unlucky split is repeated until the numbers look favourable.

The split is performed on the raw dataframe rather than on the preprocessed feature matrix. This ordering is deliberate: all subsequent preprocessing steps — median imputation, scaling, one-hot encoding, and correlation-based feature dropping — are fitted only on the training partition, and the test partition is transformed using the parameters learned on the training data. Doing it the other way around (fitting on the full dataset and splitting afterwards) would leak information from the test set into preprocessing, which would in turn produce overly optimistic estimates of generalisation error. With roughly 21,700 listings in the curated dataset, the 80/20 split leaves around 17,400 training observations and 4,300 test observations — both large enough to produce stable parameter estimates during training and a reliable estimate of generalisation error on the held-out test set.

One practical issue arises from this ordering. Because the one-hot encoder is fitted only on the training partition, any category that happens to appear exclusively in the test set — for example a `house_type_id` value with very few listings, all of which fall in the test split — will not have a corresponding column in the training feature matrix, and any training-only category will not appear in the test matrix. Both situations are handled explicitly: missing columns in the test set are added and filled with zeros, and unexpected extra columns are dropped. The two matrices are then reindexed to the same column order before any model is trained or evaluated, which guarantees that the test features are aligned with the schema the model was fitted on.

The held-out test set is not touched again until the very end of the pipeline, after model selection and hyperparameter tuning have been completed using nested cross-validation on the training data alone (Section 5.3). This separation ensures that the test-set performance (not the results reported in Section 5.3) is an honest estimate of how the final model would generalise to unseen Madrid listings.

### 3.4 Anomaly Detection on Train set

Isolation Forest (12) identifies outliers by constructing isolation trees through recursive random partitioning of the feature space. Observations that require fewer partitions to isolate exhibit anomalous behaviour and receive lower `decision_function` scores. We removed the 50 observations with the lowest anomaly scores from the training set to mitigate potential data quality issues.

This conservative approach removes 0.3% of the training set (50 rows/17,000 total), targeting only the most extreme anomalies while preserving the distribution and natural variance in property prices and structural characteristics across the Madrid market.

The test set remained untouched, as outlier removal must not be informed by evaluation data to prevent optimistic bias in performance estimates.

### 3.5 Log-Transformation of Skewed Target Distribution

The target variable (`buy_price`) exhibited right skewness ( $\text{skewness} = 3.49$ ), with a long tail extending to luxury properties above €2M. Raw-scale prices violated the normality assumption underlying least-squares and gradient boosting loss functions, risking disproportionate gradient weight on extreme luxury outliers despite their rarity in the data set. We applied log-transformation via  $y = \log(1 + \text{price})$  to both training and test targets prior to model fitting.

This transformation compresses the price range, rendering the distribution approximately Gaussian (post-transformation skewness = 0.46). The log-scale provides three benefits: (1) stabilised variance across price segments, preventing high-value transactions from dominating loss gradients; (2) improved numerical stability in boosting algorithms, enabling finer regularisation; and (3) better diverging learning curves.

### 3.6 ML algorithms

To evaluate predictive performance across different approaches, we selected five regression models from different areas of supervised learning. The idea was to compare models that work in noticeably different ways, rather than testing several small variations of the same method.

**Linear Regression.** Ordinary least squares (10) is included as the simplest possible baseline. It assumes the target is a linear combination of the features with no penalty on the coefficients, which makes it fast to fit and trivial to interpret, but vulnerable to instability when features are correlated, exactly the situation produced by one-hot encoding the district and house-type variables. Its role here is to set the lower bound of usable performance that any more sophisticated method needs to beat.

**Ridge Regression.** Ridge regression (8) extends ordinary least squares with an  $L_2$  penalty on the coefficient vector, which shrinks correlated coefficients toward each other and stabilises estimation when features are collinear. After one-hot encoding the district and house-type variables, the feature space contains many correlated columns, which makes a regularised linear model a more sensible baseline than plain linear regression. The model is fast to fit and easy to interpret, making it a useful reference point for the other methods.

**ElasticNet.** ElasticNet (19) combines  $L_1$  and  $L_2$  regularisation, performing both coefficient shrinkage and variable selection in a single model. The  $L_1$  component is particularly useful in this setting because many of the one-hot encoded dummy variables (districts, house types) are expected to carry little predictive value individually and can be driven to exactly zero. The mixing ratio between the two penalties is treated as a hyperparameter and tuned during cross-validation.

**Random Forest.** Random Forest (2) is the bagging counterpart to the boosting methods described below. It fits a large number of decision trees independently, each on a bootstrap sample of the training data and using a random subset of features at every split, and then averages their predictions. The key mechanism is variance reduction: individual trees have high variance but low bias, and averaging many decorrelated trees stabilises predictions without sacrificing their ability to capture non-linearities and feature interactions. It is included here as a baseline for ensemble methods that grow trees in parallel rather than sequentially.

**Gradient Boosting.** Gradient boosting (6) fits an ensemble of decision trees sequentially, where each new tree is trained to reduce the residual error of the ensemble built so far. Unlike bagging methods such as Random Forest, where trees are grown independently in parallel, gradient boosting is an inherently additive procedure that gradually corrects its own mistakes. The implementation used here is the classical `GradientBoostingRegressor` from scikit-learn, included as a reference point against which the more modern boosting libraries described below can be compared.

**XGBoost.** XGBoost (4) is an optimised implementation of gradient boosting that introduces several improvements over the classical algorithm: a regularised objective that penalises tree complexity directly, a histogram-based approximate split-finding algorithm, native handling of missing values, and built-in  $L_1$  and  $L_2$  regularisation on leaf weights. These changes make it considerably faster and less prone to overfitting than classical gradient boosting on structured tabular data, and it has become a standard baseline for tabular regression tasks.

**LightGBM.** LightGBM (9) is a second modern gradient-boosting implementation, included alongside XGBoost as an alternative point of comparison. The following explanation is more detailed because our experiments suggested it as the best-performing model. LightGBM, developed at Microsoft, addresses two core efficiency bottlenecks of conventional implementations: training speed and memory consumption. LightGBM grows trees leaf-wise — always splitting the leaf with the highest information gain — which reduces loss more aggressively per iteration but introduces a higher risk of overfitting on small datasets. Two algorithmic innovations distinguish LightGBM from prior implementations. First, Gradient One-Side Sampling (GOSS) avoids scanning all data instances for every split. Instances are sorted by the absolute value of their gradient: those with large gradients (high residual error, hence more informative) are retained in full, while a stratified random sample of low-gradient instances is kept and upweighted. This preserves the data distribution while concentrating computational effort on under-trained observations. Second, Exclusive Feature Bundling (EFB) reduces dimensionality by bundling mutually exclusive sparse features — features that rarely carry non-zero values — into composite features. This converts the bundling problem into a graph colouring problem. Both innovations operate on top of a histogram-based split-finding algorithm, which discretises continuous feature values into a fixed number of bins. While this introduces a loss of precision compared to pre-sorted exact-split algorithms, it counterintuitively improves generalisation: weak, noise-driven splits that would differentiate observations by marginal feature differences are collapsed into the same bin, effectively acting as implicit regularisation. The paper reports training speedups of up to 20x over conventional gradient boosting decision trees implementations at near-identical accuracy levels.

Overall, these seven models cover a range of regression approaches: linear methods both unregularised (Linear Regression) and regularised (Ridge and ElasticNet), bagging (Random Forest), classical gradient boosting (GradientBoostingRegressor), and two modern gradient-boosting implementations (XGBoost and LightGBM). Comparing them provides a broader view of the trade-offs between predictive performance, training cost, and interpretability across the model families relevant to tabular regression.

## 4 Evaluation Methodology

This section presents an overview over the metrics which are applied to evaluate the models in a statistical and interpretable way.

### 4.1 Choice and Justification of Performance Metrics

The evaluation of regression models for real estate valuation requires metrics that are statistically rigorous and practically interpretable. Different metrics capture fundamentally different properties of the errors, and no single measure dominates across all desirable criteria (1). Accordingly, five complementary metrics were employed throughout this study. Their mathematical definitions, statistical properties, and specific justifications for the present use case are explained

below.

#### 4.1.1 Coefficient of Determination ( $R^2$ )

The coefficient of determination is defined as

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}, \quad (1)$$

where  $y_i$  denotes the observed target value,  $\hat{y}_i$  the model prediction, and  $\bar{y}$  the sample mean.  $R^2$  measures the proportion of variance in the target variable explained by the model. On the final held-out test set (unseen throughout nested CV),  $R^2$  provides a relative ranking of models under identical evaluation conditions.  $R^2$  should *not* be used as an absolute measure of accuracy, as its magnitude is determined by the variance of the evaluation set rather than by intrinsic model quality. For the Madrid residential dataset, which spans a wide price range from entry-level properties to luxury apartments,  $SS_{\text{tot}}$  is large, artificially dragging  $R^2$  down for any model that captures even coarse price variation. Accordingly,  $R^2$  is reported alongside absolute error metrics but is not used as the primary evaluation criterion.

#### 4.1.2 Mean Absolute Error (MAE)

The Mean Absolute Error is defined as:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (2)$$

Each prediction error contributes linearly to MAE, making it robust to the extreme values present in the upper tail of the price distribution.

MAE provides a more natural measure of average error magnitude than quadratic alternatives, as it is expressed in the same units as the target variable. In the present context, MAE is reported in euros, making it directly interpretable as the average absolute deviation between the predicted and observed transaction price. This aligns with industry practice in automated valuation, where prediction accuracy is typically communicated to end-users in monetary terms.

#### 4.1.3 Root Mean Squared Error (RMSE)

The Root Mean Squared Error is defined as

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (3)$$

The quadratic nature of the loss function implies that large individual prediction errors exert disproportionate influence on RMSE relative to MAE, as each error contributes with weight proportional to its own magnitude.

(3) argue that RMSE is the appropriate primary metric when errors are likely to be gaussian distributed, which is the case for the real estate regression task, if the target variable (*buy\_price*) is indeed log-transformed. Additionally large errors carry disproportionate real-world costs. In residential mortgage underwriting and collateral valuation, this is true. RMSE is complementing

MAE by providing sensitivity to the tail behaviour of the error distribution, particularly relevant for the luxury segment of the Madrid market.

#### 4.1.4 Median Absolute Error (MedAE)

The Median Absolute Error is defined as

$$\text{MedAE} = \text{median}(|y_1 - \hat{y}_1|, \dots, |y_n - \hat{y}_n|). \quad (4)$$

MedAE achieves the maximum breakdown point of 0.5 among location estimators, meaning it remains a valid measure of central tendency even when up to 50% of predictions are arbitrary. Unlike MAE, whose value is pulled upward by luxury outliers regardless of model performance on the bulk of the distribution, MedAE isolates prediction errors.

The Madrid residential price distribution exhibits a pronounced right skew: the majority of transactions occur in the € 100,000–€ 500,000 range, while a small number of luxury properties extend beyond € 1,000,000. (14) note that pricing models for heterogeneous real estate markets are particularly susceptible to mean-based error inflation from the luxury segment.

#### 4.1.5 Mean Absolute Percentage Error (MAPE)

The Mean Absolute Percentage Error is defined as

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100, \quad (5)$$

where  $y_i > 0$  for all  $i$ , a condition guaranteed by the positive nature of property prices.

MAPE is the only standard metric in this set that is scale-invariant: e.g. a prediction error of € 20,000 on a € 100,000 property (20% MAPE) is treated differently in magnitude from the same absolute error on a € 1,000,000 property (2% MAPE). In the present regression context, MAPE enables direct comparison of model accuracy across Madrid’s price segments using a common relative scale.

#### 4.1.6 Metrics Summary

All metrics reported in subsequent sections are computed on the log-inverse-transformed scale, so that errors are expressed in euros rather than in units of  $\log(1 + \text{price})$ . This transformation is essential for interpretability.

## 5 Performance and Results

In this section the model evaluation scores are presented. Also the related feature importance as well as an interpretation of the performance constrains the models have faced. Finally we compare them against prediction capabilities of domain experts.

### 5.1 Success Criteria

Model performance is evaluated against the following metrics (see 4.1):



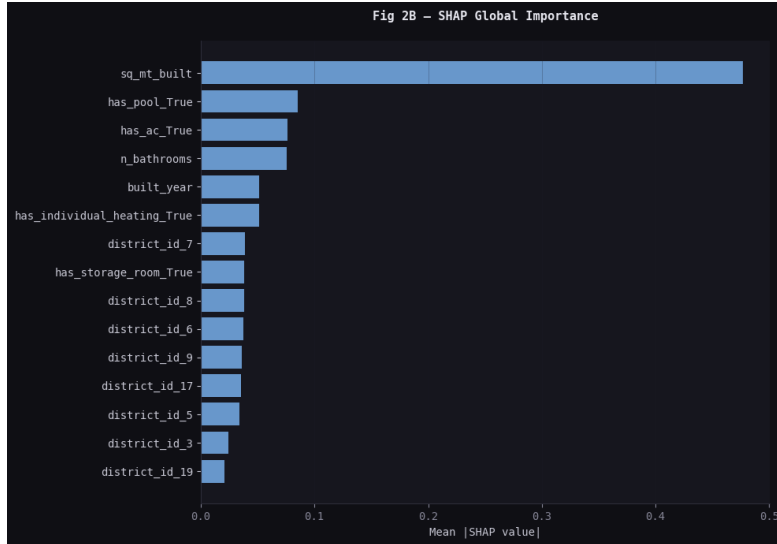


Figure 5: Shap Global Importance

- **Root Mean Squared Error (RMSE):** Commonly used in machine learning regression task, sensitive to large errors or outliers, making it useful when large errors are undesirable.
- **Coefficient of determination ( $R^2$ ):** The proportion of variance in property prices explained by the fitted model.
- **Mean Absolute Error (MAE):** This metric is directly interpretable to real estate practitioners and lenders, enabling clear communication of expected valuation accuracy in monetary units.
- **Mean Absolute Percentage Error (MAPE):** A target of MAPE  $< 7.5\%$  aligns with the empirically observed tolerance range of 5–10% reported for human appraisers in hedonic pricing studies (5), providing a domain-grounded benchmark for acceptable automated valuation performance.
- **Median Absolute Error (MedAE)** Isolates typical prediction error on normal properties, unaffected by the long tail of luxury outliers that dominate MAE.

The five metrics collectively ensure that model performance is assessed along statistical (variance explained), practical (monetary accuracy), professional (appraisal-standard) and sensitivity (squared error) dimensions.

## 5.2 SHAP-based feature importance

We computed SHAP (Shapley Additive Explanations) values on the test set as the main XAI analysis, alongside a Partial Dependence Plot for `sq_mt_built` as a global sanity check; the PDP confirmed the expected monotone relationship and is not shown separately. SHAP assigns each feature, for each individual prediction, an additive contribution to the model’s output relative to the average prediction. Summing the contributions of all features for a given listing reconstructs the model’s prediction exactly, which means SHAP values are not just a ranking but an explanation of where each predicted price comes from. Compared to the built-in feature-importance score of tree ensembles of sklearn, SHAP is less biased toward high-cardinality variables and provides directional information (whether a feature pushes the prediction up or down), which is the main reason it was preferred here.

Figure 5 shows the mean absolute SHAP value of each feature, averaged across all test predictions. This quantifies how much each feature contributes to the model’s output on average, regardless of direction. The built surface `sq_mt_built` dominates by a wide margin, with an average impact roughly five times larger than the next most influential feature. Below it, a cluster of amenity flags (`has_pool`, `has_ac`) appears alongside the structural variables `n_bathrooms` and `built_year`, after which the ranking is filled out by a series of district dummies. This ordering is broadly consistent with prior expectations: size is the dominant driver of price in any major urban market, followed by a mixture of structural quality indicators and location effects.

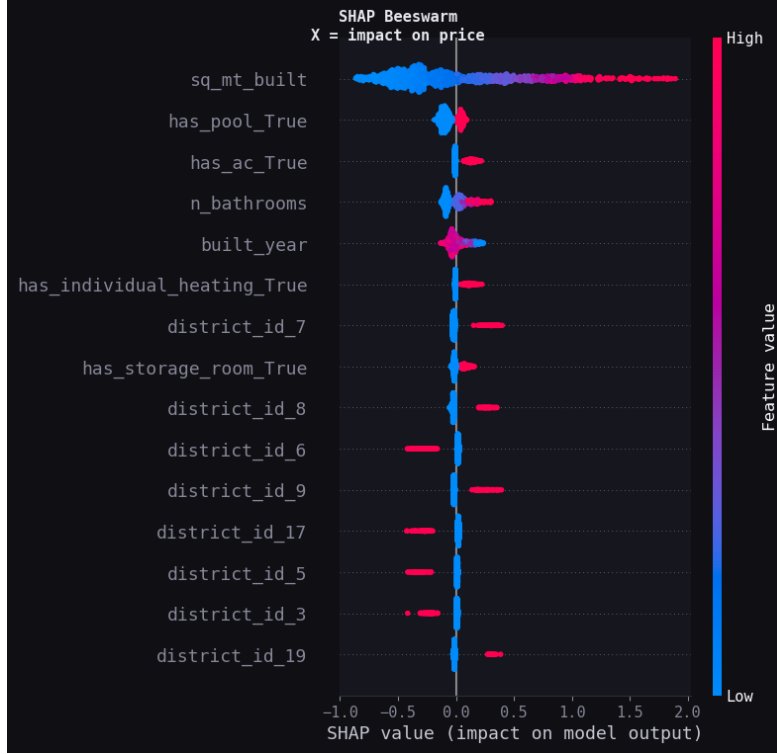


Figure 6: Shap Beeswarm

Figure 6 provides a more detailed view. Each dot represents a single test prediction; its horizontal position gives the SHAP value (i.e. the impact on the predicted price), and its colour encodes the value of the feature itself (blue for low, red for high). For `sq_mt_built`, the relationship is the expected monotone one: low values (blue) push predictions strongly downward, while high values (red) push them strongly upward. The amenity flags behave as expected: when a property has a pool, air conditioning, individual heating, or a storage room (red), its predicted price is pushed upward, while their absence (blue) has a comparatively muted effect.

The district dummies show a clearly directional pattern. Some districts (for example `district_id_7`, `district_id_8`, `district_id_9`, `district_id_19`) contribute positively to the predicted price when active, while others (`district_id_6`, `district_id_17`, `district_id_5`, `district_id_3`) contribute negatively. At first glance, this appears inconsistent with the median-price ranking in Figure 4, since some of the negative-contribution districts have very high medians. This apparent contradiction is taken up in Section 5.8.

Taken together, the SHAP analysis confirms that the model relies on economically sensible features: structural attributes, amenities, and location dummies dominate the ranking, with no sign of spurious or leakage-like features. This gives some additional confidence in the test-set

performance reported in Section 5.3.

### 5.3 Performance and Model interpretation

This section shows the over all reported metrics from the experiments. Note that the coefficient of determination ( $R^2$ ) is evaluated on log-transformed prices to preserve numerical stability and comparability, while all other error metrics (MAE, MedAE, RMSE, MAPE) are reported in non-transformed euro terms to enable direct business interpretation.

Table 2: Model Selection Results: Nested Cross-Validation Performance. Five-fold outer CV and three-fold inner CV via RandomizedSearchCV (except for the Linear Regression model). Metrics reported in euros on test scale after inverse log-transformation.

| Model             | $R^2$ | MAE (€) | MedAE (€) | RMSE (€) | MAPE (%) |
|-------------------|-------|---------|-----------|----------|----------|
| Linear Regression | 0.912 | 163,163 | 50,390    | 459,055  | 20.41%   |
| Ridge             | 0.903 | 161,255 | 52,457    | 396,455  | 21.38%   |
| ElasticNet        | 0.911 | 164,429 | 50,539    | 464,218  | 20.55%   |
| RandomForest      | 0.918 | 132,155 | 48,620    | 299,292  | 19.57%   |
| GradientBoosting  | 0.951 | 107,687 | 38,203    | 246,470  | 15.05%   |
| XGBoost           | 0.951 | 107,057 | 37,234    | 249,491  | 14.95%   |
| LightGBM          | 0.945 | 104,956 | 36,086    | 237,751  | 14.58%   |

The comparison using nested cross-validation revealed performance differences between the model families, each of which is based on its own learning mechanism. Understanding these differences explains why certain architectures perform better on tabular datasets. The subsequent evaluation of XGBoost and LightGBM on the test dataset (these two models, as they achieved the lowest MAE in the previous experiments) yielded the following results: XGBoost RMSE: 243,156 and LightGBM RMSE: 239,219, which shows similar performance to that in the nested cross-validation experiment. For comparison, the Linear Regression Model got to a RMSE of 465,775 and the Random Forest Model 270,744 RMSE.

**Linear models (Ridge, ElasticNet):** Ridge regression minimises its error by imposing a L2 penalty on coefficients. This mechanism assumes price is a weighted linear combination of features—defensible for large continuous features like square meters but insufficient for capturing interactions, shrinking weak coefficients to zero. On this dataset, ElasticNet underperformed Ridge (Table 2) because the L1 sparsity eliminated informative features whose individual correlations were weak but whose interactions mattered.

**Decision trees (RandomForest):** Random Forest (RF) was evaluated as a baseline for bagging ensemble models. A Random Forest averages  $B$  independent trees, each trained on a bootstrap sample with random feature subsets. The key mechanism is *variance reduction*: an individual tree has high variance but low bias (captures true non-linearities). Averaging uncorrelated trees stabilises predictions. On this dataset, RandomForest achieved RMSE 299,292 ( $R^2$  0.918), a substantial jump over linear models. The improvement stems from feature interactions that can be found by splitting instead of finding a mathematical relationship between them. However, RandomForest’s performance plateaued because averaging trees produces errors that are binned rather than smoothed out. For continuous features like size, this discretisation leaves residual error.

**Gradient boosting (GradientBoosting, XGBoost, LightGBM):** These models sequentially build trees, each correcting the residuals of the previous ensemble. The mechanism is *bias reduction*: instead of averaging high-variance models (RF), boosting fits weak learners to residuals, gradually reducing underfitting.

The three boosting implementations differ in tree construction strategy:

- **GradientBoosting:** Grows trees level-wise (breadth-first), building complete levels before proceeding deeper. This ensures balanced trees but is less efficient at isolating leaf splits. Achieved RMSE 246,470 ( $R^2$  0.951).
- **XGBoost:** Grow trees leaf-wise, selecting the globally best split at each step. This concentrates splits where they reduce loss most, improving data efficiency. Achieved RMSE 249,491 ( $R^2$  0.951), a slight decrease in performance against GradientBoosting.
- **LightGBM:** In the present study, LightGBM achieved the best overall performance across all evaluated models, with an RMSE of 237,751 and an  $R^2$  of 0.945. This represents a 4.7% relative improvement in RMSE over XGBoost, while the  $R^2$  scores remained comparable. The RMSE improvement is consistent with the paper’s efficiency claims: histogram binning suppresses overfitting on noisy feature relationships, which in a regression context that leads directly into lower prediction error. The absence of a corresponding  $R^2$  gain suggests that both models explain a similar proportion of variance, but LightGBM’s implicit regularisation reduces large prediction errors more effectively — a distinction that RMSE, as a squared-error metric, is sensitive to.

## 5.4 Performance Border and Domain Constraints

The final model (LightGBM) achieves an error of  $RMSE = 237,751$  and  $MAPE = 14.58\%$  in the nested cross-validation run, representing substantial improvement over the baseline ( $RMSE$  459,055,  $MAPE$  20.41%) by almost 50% on RMSE. However, a performance plateau at around  $MAE$  105,000 across multiple model architectures suggests an underlying error floor determined by data limitations rather than algorithmic choice.

**Sources of irreducible error:**

- **Interior quality and condition.** The dataset captures interior condition only as the binary `is_renewal_needed` flag. In reality, interior finish spans a wide spectrum — from properties in need of full renovation, through standard finishes, up to luxury designer interiors — and collapsing this range into a single yes/no signal discards substantial price information (14).
- **Location granularity and Environmental factors.** Location enters the model at two levels, namely a coarse `district_id` dummy and a fine-grained `subtile` dummy at the barrio level (roughly 140 distinct barrios after one-hot encoding, see Section 3.2.5). The feature set does not, however, capture sub-barrio variation, including differences between adjacent streets, the condition of individual buildings on the same block, and orientation onto a quiet courtyard versus a busy avenue. Environmental amenities of the kind used in hedonic price models, such as proximity to parks, schools, or transport stops, are likewise absent from the dataset. Both omissions contribute to the error floor and cannot be recovered by a different choice of algorithm.
- **Time on market.** The number of days a property has been listed is not recorded. A

long time on market is a strong signal of an over-asking listing, seller motivation, or a less desirable property; without it, the model treats a listing that has just been posted identically to one that has been sitting unsold for months.

- **Market timing.** The dataset is a single cross-sectional snapshot, so the model cannot distinguish properties listed in a hot market from those listed in a slow one, or capture broader cyclical trends. Even within the short scraping window, individual listings may not have been refreshed since they were first posted and therefore reflect older market conditions.
- **Listing price and transaction price.** The target variable is the asking price set by the seller, not the final transaction price. The gap between the two ranges from 0% to 15% depending on market conditions, seller motivation, and negotiation skill (11). This systematic noise in the target variable sets a lower bound on achievable accuracy that no choice of model can overcome.
- **Unobservable heterogeneity.** Unique properties — heritage buildings, properties with special legal status, plots with restricted development rights — have valuations determined by factors that no structured dataset can capture: historical significance, community status, seller urgency, emotional attachment.

Each of these omissions is correlated with price and with features that *are* included in the model (built surface, district, construction year), which means their effects are silently projected onto the available predictors. The practical consequences are model-wide: properties on the best streets within a given district are systematically undervalued, ageing or poorly maintained listings are valued as if they had standard finishes, and short-listed and long-listed properties at the same asking price are treated identically. None of these errors are visible in the test-set metrics, because the test set inherits exactly the same blind spots as the training set — which is the key point that connects the technical discussion here to the ethical concerns raised in Section 6.1.

## 5.5 Impact of Log-Transformation on Model Performance

The target variable was evaluated in two forms: raw price (euros) and log-transformed price 3.5. Table 3 compares LightGBM performance under both circumstances on the test set. The model had no hyperparameter tuning.

Table 3: LightGBM test performance: raw vs log-transformed target. Log-transformation yields modest but consistent improvements across all metrics, suggesting the model learns better from the compressed, normalized target distribution.

| Target Scale       | R <sup>2</sup> | MAE (€)   | MedAE (€) | RMSE (€)  | MAPE (%) |
|--------------------|----------------|-----------|-----------|-----------|----------|
| Raw price          | 0.877          | € 121,273 | € 51,216  | € 250,967 | 19.24    |
| Log-transformed    | 0.885          | € 114,756 | € 42,705  | € 242,619 | 16.23    |
| <b>Improvement</b> | 0.9%           | 5.37%     | 16.62%    | 3.33%     | 15.64%   |

Log-transformation of the target variable produced consistent performance gains across all metrics: R<sup>2</sup> increased from 0.877 to 0.885 (+0.9% relative), while absolute errors decreased by 5.4% (MAE), 16.6% (MedAE), and 3.3% (RMSE). The improvements likely stem from two factors: (1) log-transformation compresses the right-skewed price distribution, making it more Gaussian

and reducing the leverage of luxury outliers (€800k+) on loss gradients; and (2) the normalized scale provides more stable gradient estimates during boosting, enabling finer regularisation tuning. These gains justify the use of log-scale targets.

## 5.6 Training Stability and Convergence Analysis

Two internal validation metrics were tracked during a newly fitted LightGBM model on training and test data to assess model stability and convergence behavior. Because of time constraints the a less complex LightGBM model was used for this tested which performed around 9% worse than the best more complex LightGBM model. Figure 7 displays the evolution of (A) raw-scale RMSE and (B) log-scale RMSE across 100 boosting iterations, each measured on two independent validation folds (`valid_0` and `valid_1`).

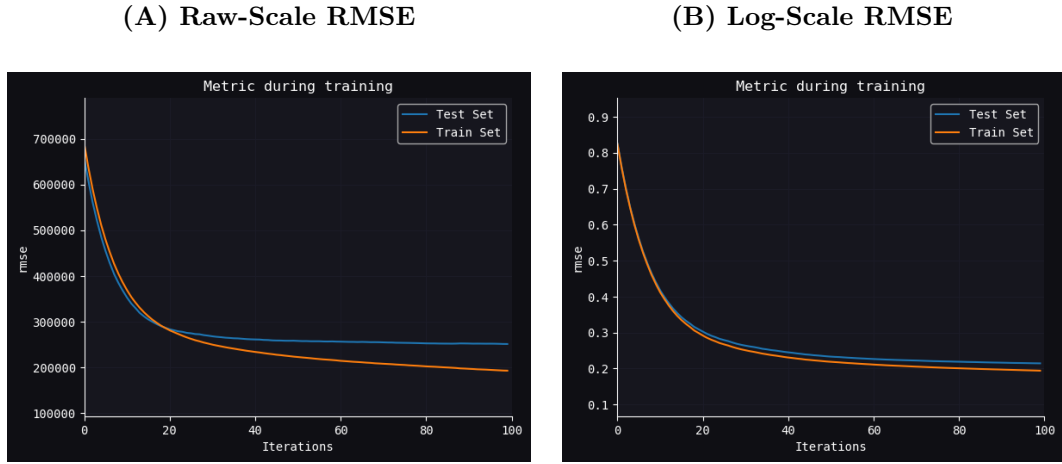


Figure 7: Training convergence comparison: raw (left) vs log-transformed (right) target scales. Blue and orange lines represent independent validation folds. Log-scale training exhibits tighter fold agreement and earlier convergence plateau, indicating more stable regularisation and reduced variance across CV splits.

**Raw-scale RMSE (Panel A):** Validation metrics diverge notably across folds—`valid_0` plateaus at €250k while `valid_1` reaches €200k by iteration 100. This €50k gap between folds indicates unstable predictions across the validation dataset and suggests the model fitting patterns that vary by fold. The plateau beginning around iteration 40 implies boosting rounds beyond this point provide diminishing returns, motivating early stopping.

**Log-scale RMSE (Panel B):** The two validation folds track nearly identically throughout training, with both converging to RMSE around 0.20 (equivalent to €50k–€70k). The tight fold agreement demonstrates that log-transformation yields more stable, generalizable predictions. Convergence occurs earlier and the error is lower, confirming that the normalized target scale provides superior regularisation properties. The absence of fold divergence suggests reduced overfitting risk and more reliable cross-validation estimates.

**Interpretation:** Log-transformation not only improves absolute test performance but also stabilises the learning during training. The tighter validation curves in Panel B indicate more robust hyperparameter tuning and reduced sensitivity to fold assignment, a desirable property for production deployment.

## 5.7 Data Size Effects on Model Architecture (Boosting and Bagging)

In *k-Fold* evaluation, Random Forest achieved a **MedAE** of 38,173, outperforming XGBoost (MedAE: 40,210). Under *nested cross-validation*, however, the ranking reverses: XGBoost achieves a MedAE of 46,587 compared to 63,125 for Random Forest. This divergence is attributable to two distinct mechanisms. First, standard *k-Fold* cross-validation with default hyperparameters masks Random Forest overfitting, because training and test folds share the same data distribution. Nested cross-validation corrects for this by tuning hyperparameters on inner-fold data and evaluating on a truly held-out outer fold, yielding more conservative but unbiased performance estimates. Second, because of bootstrapping, the effective subsample per learner in Random Forest is reduced further, as each tree is trained on the already-reduced training set.

## 5.8 Role of district in price predictions

As Figure 4 and Figure 6 point out contrastingly the appearance of high-priced districts pushes down the predicted price by the model. This does not make intuitive sense in the first place. Our hypothesis is that the appearance of cheaper or more expensive districts as a feature (which are One Hot encoded) functions as a corrective feature. This is supported by Figure 8, which shows that especially expensive but also cheaper districts push the error up. Concluding, this explains why expensive districts drag down the price and the other way around.

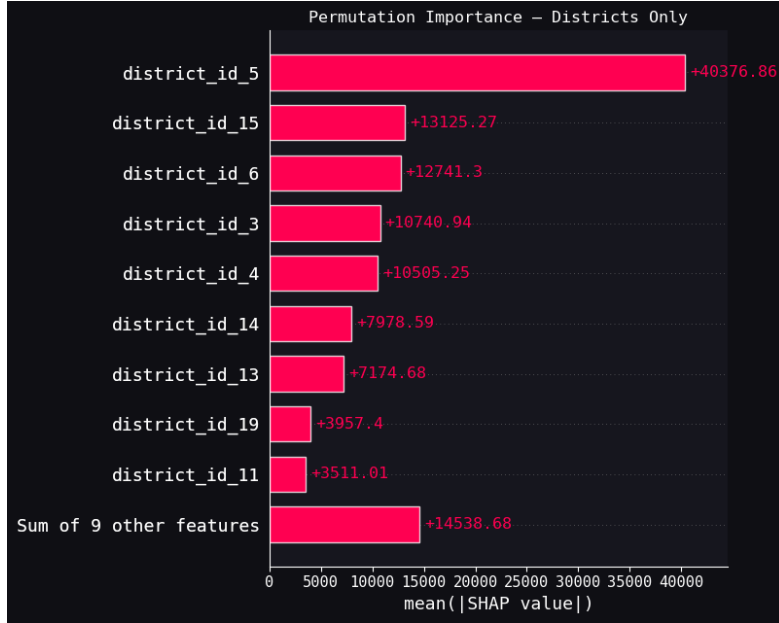


Figure 8: The importance of districts as an error correction feature

## 5.9 Statistical testing via Paired t-tests

Four regression models were compared using 10-fold cross-validation with RMSE as the evaluation metric. The models were evaluated without hyperparameter tuning. For each fold, each model produced an RMSE estimate on the held-out validation set, yielding 10 paired observations per model. Paired t-tests were conducted on these fold-level errors to determine whether performance differences are statistically significant beyond random variation. The test statistic  $t = \frac{\bar{d}}{s_d/\sqrt{n}}$ , where  $\bar{d}$  is the mean difference in RMSE across folds and  $s_d$  is the standard deviation of those differences, follows a t-distribution with  $n - 1 = 9$  degrees of freedom under the null hypothesis

that the models have equal expected error. Rejection of the null hypothesis ( $p < 0.05$ ) indicates that the observed RMSE difference is replicable across folds and not attributable to fold-specific noise or chance. The paired design, in which all models are evaluated on identical folds, reduces confounding from fold difficulty and increases statistical power. Results are visualised as box plots sorted by mean RMSE, with individual fold scores overlaid to reveal fold-to-fold stability. The same procedure was applied to the tuned model evaluation (see Figure 10), with the only modification being a reduction to five outer folds while all remaining parameters were kept unchanged.

As Figure 9 shows XGBoost, Random Forest and LightGBM are almost identical in terms of their performance while only the Decision Tree falls behind, if the models are not tuned on their hyperparameters. The t-test suggests that between XGBoost, Random Forest and LightGBM, no significant differences were detected, and only the Decision Tree performs significantly worse.

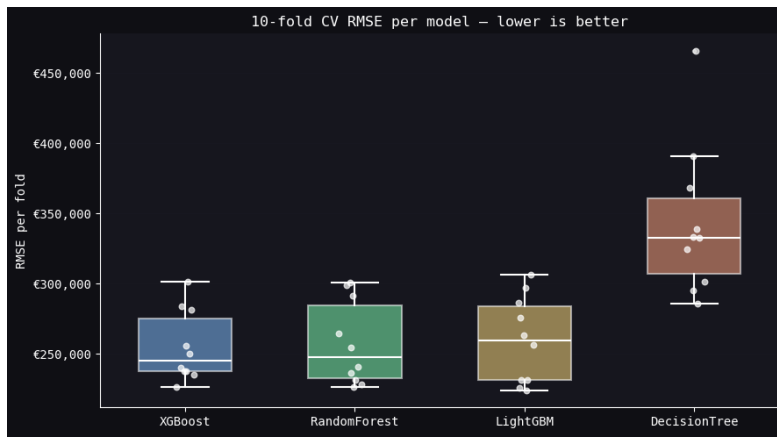


Figure 9: Paired t-test on regression models

As Figure 10 illustrates, when models are evaluated at their tuned hyperparameter configurations — and therefore operate at peak performance — the paired t-test yields a statistically significant result, identifying LightGBM as the superior model. This stands in contrast to the default-parameter comparison, where differences were indistinguishable.

One plausible explanation for this shift lies in how each algorithm responds to regularization. LightGBM’s leaf-wise growth strategy allows it to allocate model complexity precisely where the data demands it — concentrating splits on the highest-gain regions. Hyperparameter tuning amplifies this advantage: once `num_leaves`, `min_data_in_leaf`, and the regularization terms are optimized for the Madrid dataset’s specific structure LightGBM can exploit these patterns more efficiently than XGBoost’s depth-wise growth, which distributes splits more uniformly across the tree regardless of local gain.



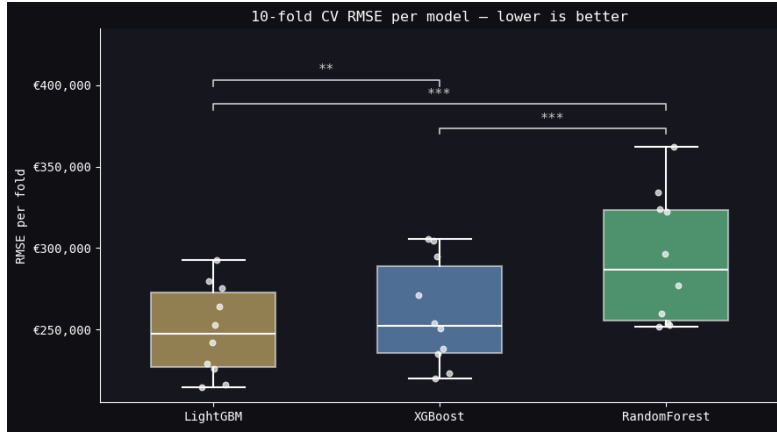


Figure 10: Paired t-test on regression models with tuned hyperparameters

## 5.10 Answering the research questions

This study addresses two primary research questions (see section 1.2). Firstly (1): Which machine learning model achieves the best predictive performance? And secondly (2): Is the improvement significant, and does it therefore justify greater complexity and, potentially, a black-box model?

(1) The best performing model for this regression task is LightGBM (5.3), which achieved (with hyperparameter tuning) a MAPE of 14.58% and a RMSE of 237,751. Not negligible is as well that LightGBM took by far the longest amount of training and testing time. The time-performance trade-off should be evaluated if the model would go into production. The project proposal defined a target of  $\text{MAPE} < 7.5\%$ , motivated by empirical evidence suggesting that human appraisal error typically falls within a range of 5% to 10% (18). This threshold was not achieved by any of the evaluated models.

(2) Section 5.9 provides the necessary information and shows that LightGBM has, without hyperparameter tuning no statistical significantly better performance against XGBoost and RandomForest. These three models can be all considered at least gray-box models because of their internal complexity. It is reasonable to use LightGBM as a best performing model and prefer it over the interpretable Decision Tree. However, the use of explainable AI (XAI) techniques should play a key role when deciding whether to use LightGBM.<sup>1</sup>

## 6 Ethical Considerations

Any automated valuation model that influences real-world financial decisions carries ethical implications that extend beyond predictive accuracy. A model that performs well on a held-out test set may still reproduce harmful patterns present in its training data, lack the transparency needed for users to challenge its outputs, or drift silently as the underlying market evolves.

### 6.1 Data Bias

The course identifies five common types of data bias: response bias, selection bias due to feedback loops, bias due to system drift, omitted variable bias, and societal bias. Below are biases that

<sup>1</sup>As suggested by (4), further investigation for better performance could go in the direction of creating ensemble methods with boosting techniques combined with neural networks.

apply to the dataset used in this project, and each has practical consequences for how the model’s outputs should be interpreted.

**Selection bias.** The dataset consists of listings scraped from popular Spanish real estate portals over a short collection window. The population represented is therefore not “all properties in Madrid” but “properties actively advertised online during the scraping period,” excluding private sales, off-market transactions, and properties traded outside the major portals. The recorded prices are also asking prices rather than transaction prices, which means the model predicts what sellers ask for rather than what buyers pay. These selection effects are not correctable through better modelling: any algorithm trained on this dataset inherits them.

**Omitted variable bias.** The Madrid dataset is missing several features that are correlated with price and with the variables that are present, including interior quality, micro-location, environmental factors, time on market, and market timing. Each of these satisfies the textbook condition for omitted variable bias in regression: correlation with the target, and correlation with at least one included predictor. A detailed enumeration of these omissions and their effects on model behaviour is given in Section 5.4, where the same facts are framed as sources of irreducible error. The ethical concern here is the same set of omissions seen through a different lens: the model’s systematic mistakes are not random noise but predictable, patterned errors that disadvantage particular kinds of properties and, by extension, particular kinds of owners.

**Societal bias and reinforcement.** Property prices in Madrid exhibit a strong geographic gradient, with central and northern districts commanding substantial premiums over peripheral ones. This gradient is real, but it is also the historical product of decades of urban planning, public investment, and demographic sorting — not a neutral fact. A model that reproduces it and is then used to inform mortgage decisions or investment valuations risks reinforcing it: properties in lower-priced districts receive systematically lower automated valuations, which can affect the credit and capital available to their owners. This is the standard mechanism by which predictive models trained on socially patterned data perpetuate the patterns they were trained on.

## 6.2 The Five C’s

The Five C’s (Consent, Clarity, Consistency and Trust, Control and Transparency, and Consequences) are framing guidelines from the course for building responsible data products. The framework is aimed mostly at consumer-facing apps, but each component raises a real question for a model of this kind.

**Consent.** The listings were posted publicly so that buyers could see them, not so that they could be used as training data for a price model. Whether that counts as meaningful consent for the present use is not a question the original Kaggle upload addresses.

**Clarity.** A user who gets a price out of the model should understand what it actually is: an estimate of what a similar property would be *advertised* for. It is not a sale price, not an appraisal, and not a normative “true value.” Communicating that distinction matters.

**Consistency and Trust.** The Madrid market is not static — prices move, districts gain or lose desirability, and building standards change. A model trained on one snapshot will drift out

of calibration as the market evolves, which is the system drift problem the course flags. Trust therefore, depends on a clear plan for re-training and monitoring, not on a one-off deployment.

**Control and Transparency.** The SHAP analysis in Section 5.2 is the main transparency mechanism. Every prediction can be broken down into the contribution of each feature, which gives users something to push back on when they think the model has got it wrong — rather than just a single opaque number.

**Consequences.** This is the one that matters most. A wrong prediction on a test set is a statistical inconvenience; a wrong prediction in deployment can affect mortgage approvals, tax assessments, and the prices at which owners can list their homes. That asymmetry is why the model should never be the only input to a high-stakes decision, and why human appraisal should remain available when its output is challenged.

## 7 Conclusion

This project built and evaluated an end-to-end pipeline for predicting residential asking prices in Madrid, working from the Kaggle *Madrid Real Estate Market* dataset of 21,742 listings and 58 raw attributes per listing. The workflow followed the structure introduced in the course: frame the problem, explore the data, clean and preprocess it under strict leakage control, compare several model families, select and evaluate on a held-out test set, interpret what the model learned, and reflect on the ethical implications of deploying it.

The exploratory work set the agenda for everything that followed. Asking prices were heavily right-skewed, which is what motivated the log transform of the target. A simple univariate fit of price on built surface gave  $R^2 = 0.692$  (see Figure 3 top left) and district medians varied by more than a factor of three across the top ten districts – although the within-district spread grew so sharply with the median (see Figure 4) that a single district dummy was only ever going to capture an average effect rather than a precise location premium. The EDA also flagged a handful of columns that were either too sparse to be useful or would have leaked the target, motivating their removal in the preprocessing pipeline.

The preprocessing pipeline was designed around the train/test split. Only deterministic, row-local steps happened before the split; everything statistical – mode imputation for orientation flags, **False**-imputation for amenities, district-level median imputation for construction year, one-hot encoding, correlation-based feature dropping – was fit on the training partition only. Seven regression models were compared, covering (regularised) linear methods (Linear Regression, Ridge, and ElasticNet), bagging (Random Forest), and gradient boosting (GradientBoosting, XGBoost, and LightGBM).

Evaluation used five complementary metrics in parallel rather than relying on any single one. LightGBM came out best, with RMSE €237,751, MAE €104,956, MedAE €36,086, MAPE 14.58%, and  $R^2 = 0.945$  – roughly a 50% reduction in RMSE over the linear regression baseline. The boosting models as a group cleanly outperformed both regularised linear regression and Random Forest, in line with what sequential residual fitting would predict. A side-by-side comparison of raw versus log-transformed targets (Table 3) confirmed that the log scale produced lower errors. SHAP analysis showed that the model relied on economically sensible features. The initially puzzling finding that some high-median districts pushed predictions down is plausibly explained, though not definitively shown, by those district dummies acting as a corrective feature

on top of size and amenities rather than as a standalone location premium (see Section 5.8). The paired t-test on tuned models showed that LightGBM were statistically distinguishable from XGBoost, Random Forest and a single Decision Tree.

The ethical picture is closely tied to those same omissions and is worth investigating as different concerns arise in the current data landscape of the project (see 6).

Deploying the model in any decision-making context, whether for buyer or seller guidance, automated valuation screening, or input to mortgage or tax workflows, would require improvements that go beyond raw predictive accuracy. The fixed percentage band associated with the headline MAPE of Table 2 would have to be replaced by a calibrated uncertainty estimate, for example through per-district residual quantiles or LightGBM quantile regression, so that the interval reflects the heteroscedasticity visible across districts in Section 5.4. The training data is itself a constraint, since the dataset used here is a single snapshot from March 2020 and the Madrid market has moved appreciably in the years since, so a deployment in the present would have to begin with re-collection from current listings and continue with a re-training and drift-monitoring schedule consistent with the system-drift concern raised in Section 6.2. Finally, any high-stakes use would have to retain a human appraiser as the final decision-maker, and the asking-versus-transaction gap discussed in Section 5.4 and quantified by Levitt and Syverson (11) would need to be addressed either by reframing the output or by linking it to transaction-price data.

## References

- [1] Botchkarev, A. (2018). Performance metrics (error measures) in machine learning regression, forecasting and prognostics: properties and typology. *arXiv preprint arXiv:1809.03006*.
- [2] Breiman, L. (2001). Random forests. *Machine Learning*, 45:5–32.
- [3] Chai, T. and Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? — arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3):1247–1250.
- [4] Chen, T. and Guestrin, C. (2016). XGBoost: a scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- [5] Cho, M. and Megbolugbe, I. (1996). An empirical analysis of central city housing prices. *Journal of Real Estate Finance and Economics*, 13(1):59–80.
- [6] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232.
- [7] García, S., Luengo, J., and Herrera, F. (2013). *Data Preprocessing in Data Mining*. Springer, Cham.
- [8] Hoerl, A. E. and Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67.
- [9] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154.
- [10] Kuchibhotla, A. K., Brown, L. D., Buja, A., and Cai, J. (2019). All of linear regression. *arXiv preprint arXiv:1910.06386*.
- [11] Levitt, S. D. and Syverson, C. (2008). Market distortions when agents are better informed than the principal. *Review of Economics and Statistics*, 90(4):597–602.
- [12] Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE.
- [13] Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, pages 4765–4774.
- [14] Pagourtzi, E., Assimakopoulos, V., Hatzichristos, T., and French, N. (2003). Real estate appraisal: a review of valuation methods. *Journal of Property Investment & Finance*, 21(4):383–401.
- [15] Raschka, S. (2018). Model evaluation, model selection, and algorithm selection in machine learning. *arXiv preprint arXiv:1811.12808*.
- [16] Rosen, S. (1974). Hedonic prices and implicit markets: product differentiation in pure competition. *Journal of Political Economy*, 82(1):34–55.
- [17] Toktogaraev, M. (2020). *Madrid Real Estate Market*. Kaggle dataset. Available at: <https://www.kaggle.com/datasets/mirbektoktogaraev/madrid-real-estate-market>

- [18] Cho, M., & Megbolugbe, I. F. (1996). An empirical analysis of property appraisal and mortgage redlining. *The Journal of Real Estate Finance and Economics*, 13(1), 45–55. <https://doi.org/10.1007/BF00174550>
- [19] Zou, H. and Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2):301–320.

## A Appendix: Web simulation of the asking-price estimator

To complement the offline evaluation reported in Sections 5 and 6, a local web simulation was built around the saved model `best_lgbm.pkl`. The simulation is not a production deployment and is not intended as one. Its purpose is to demonstrate that the model produced by the training pipeline can be loaded, fed inputs of the kind a non-technical user would supply, and made to return predictions in human-readable form, while surfacing the engineering problems that the modelling chapter could afford to ignore.

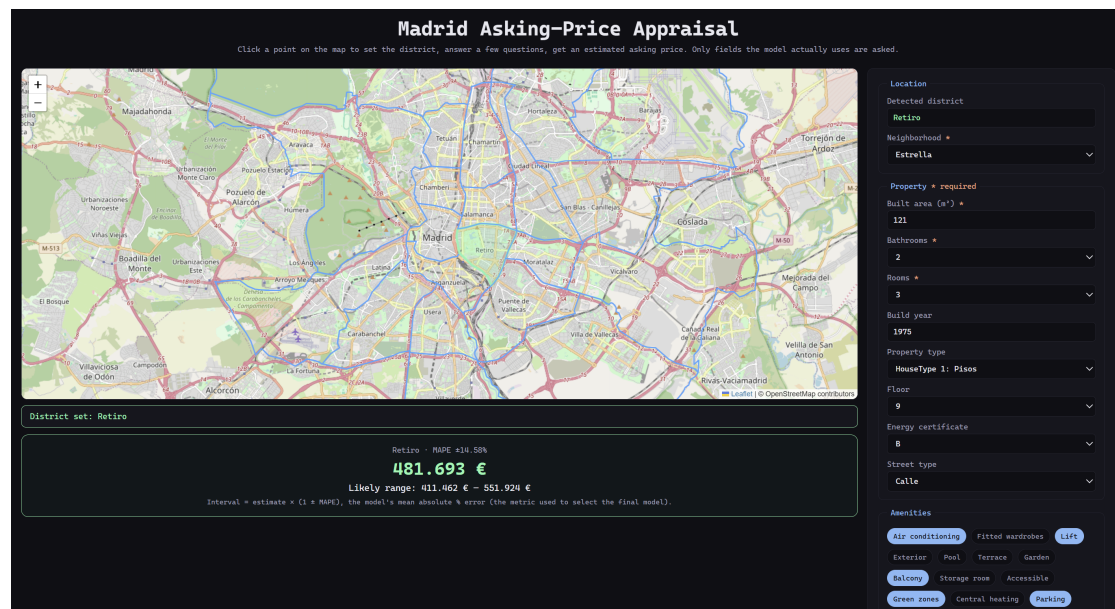


Figure 11: Web simulation of the asking-price estimator, running locally in the browser

### A.1 Purpose and scope

The user selects a point on a map of Madrid, picks a neighborhood from a dropdown that lists only neighborhoods present in the dataset for that district, fills a short form covering the structural and amenity features the model uses, and receives a point estimate of the asking price together with a percentage band. Every input requested corresponds to a feature the trained model actually uses; price-derived and rent-derived columns, which were dropped during training to prevent target leakage as described in Section 3, are never requested. The system runs entirely on the user’s machine and relies only on free, open-source components.

### A.2 Architecture

The backend is a FastAPI application that loads the model and the reference data once at startup. The model itself is read with `joblib.load` and its expected input schema is recovered from `feature_name_`. Reference data, namely the list of districts, the per-district neighborhoods,

default build-year medians, and the categorical option lists, are parsed once from the training CSV. The frontend is static HTML, CSS, and JavaScript; the map is rendered with Leaflet using OpenStreetMap tiles, and the 21 official Madrid district polygons are bundled as a public-domain GeoJSON file. Point-in-polygon resolution from a clicked coordinate to a district number uses Shapely on the backend. The `scikit-learn` version is pinned to 1.5.2 to preserve loadability of the pickled model.

### A.3 Feature reconstruction at inference time

The most delicate piece of engineering in the simulation is the construction of the 230-feature vector the model expects from a single user submission. The training pipeline applies `pd.get_dummies` with `drop_first=True`, which works correctly on a batch because the reference category for each categorical variable is identifiable as the level not represented by any one-hot column. On a single row, however, every categorical variable has exactly one value, and `drop_first` would remove it: the row would collapse silently to the all-zero reference pattern for every category, and the model would be asked to predict from a row that encoded no district, no property type, no neighborhood, and no street type.

To avoid this collapse, the simulation does not replay the notebook encoding. Instead, the function `build_X` in `madrid_model.py` constructs the feature row deterministically against the model’s own stored `feature_name_`. Numeric features are passed through, boolean flags are matched to whichever dummy form the training pipeline produced, and the six one-hot categorical families (`subtitle`, `district_id`, `house_type_id`, `floor`, `energy_certificate`, and `addr_street_type`) set the single column corresponding to the user’s selection to one, leaving the remaining columns at zero. The reference or unknown level is represented by the all-zero pattern, which is precisely what the training pipeline produced on rows whose category was the dropped reference level. The resulting feature vector is mathematically identical to what the notebook pipeline would have produced for the same listing, and the prediction is converted back to euros through `expm1`, inverting the `log1p` transformation applied to the target during training.

### A.4 User interaction, uncertainty, and limitations

The simulation enforces that the four strongest predictors, namely district, neighborhood, built area, and number of bathrooms, are always provided. Districts with no dataset listings are rendered in grey on the map and are not selectable, since the model has not seen any training examples from them. Optional fields fall back to defaults learned from the training data, with `built_year` falling back to the district median when not supplied. A small JSON API exposes the predict endpoint as well as the form metadata and the district polygons, and returns a structured response containing the point estimate, the band endpoints, and the district identification.

The percentage band around each prediction is a fixed multiplicative interval,  $\text{point} \times (1 \pm \text{MAPE}/100)$ , where MAPE is the constant 14.58 per cent reported in Table 2. Three properties of this band are worth noting. First, the band is symmetric in percentage terms but asymmetric in euros, which is consistent with the log-scale training objective. Second, the band is identical in width regardless of district, building size, or property type, even though the residual distribution from Section 5 is plainly not homoscedastic across these strata. Third, the band is therefore not a calibrated prediction interval. A more faithful representation of uncertainty, for instance per-district empirical residual quantiles or LightGBM quantile regression, would replace the fixed

band and is left for future work.

Three further limitations follow from the prototype framing. The address-derived features `addr_has_number` and `addr_number_bucket` cannot be reconstructed from the form because the user does not provide a complete street address, so the simulation defaults to the no-number pattern, which is the most common case in the training data and the closest approximation available given the form schema. The model file is required at startup; if it is missing, the API remains reachable for diagnostic purposes but the predict endpoint returns a 503 response rather than a silently generated number. Finally, the simulation provides no authentication, no rate limiting, and no audit log, which is appropriate for demonstration and for teaching but not for any use that influences a real transaction.